Backpropagation Algorithm

Ruslan Gainetdinov

December 8, 2025

1 Popis problému

Cieľom podúlohy bolo implementovať algoritmus **backpropagation** bez použitia knižníc typu PyTorch alebo TensorFlow, ktoré obsahujú automatické diferencovanie. Celá implementácia bola postavená výhradne na `NumPy`.

Následne bola úloha otestovať sieť na logických problémoch **AND**, **OR** a **XOR**, pričom každý z nich predstavuje binárnu klasifikáciu.

Úlohou bolo:

- vyskúšať architektúry s **jednou** aj **dvoma skrytými vrstvami**,
- použiť **momentum aj bez momentum**,
- pre problém XOR vyskúšať **minimálne dve hodnoty learning rate**.

Cieľom dokumentácie je porovnať správanie siete za rôznych nastavení a analyzovať priebeh učenia.

2 Metodika – Backpropagation

Implementovaný backpropagation pozostáva z týchto komponentov:

Vrstvy

- **Linear**: vykonáva výpočet
$$y = Wx + b.$$
Ukladá si vstup aj gradienty pre spätný prechod.
- **Aktivačné funkcie**:
  - Sigmoid
  - Tanh
  - ReLU

  Každá funkcia implementuje aj deriváciu.
- **MSE** – Mean Squared Error:
$$L = \frac{1}{N} \sum (y - t)^2.$$

Model

Model je tvorený zoznamom vrstiev – ide o jednoduchú *Sequential* architektúru, kde:

$$\begin{aligned} \texttt{forward():} &\quad x_{i+1} = f_i(x_i), \\ \texttt{backward():} &\quad g_i = f_i'(g_{i+1}), \\ \texttt{update():} &\quad W \leftarrow W - \eta(\nabla W + m \cdot v). \end{aligned}$$

Momentum urýchľuje zostup gradientu a stabilizuje konvergenciu.

3 Nastavenie experimentov

Boli testované tieto kombinácie:

Architektúry

- 2–4–1 (jedna skrytá vrstva)
- 2–4–4–1 (dve skryté vrstvy)

Momentum

- Momentum = 0.0
- Momentum = 0.3

Learning rate – len pre XOR

- $\eta = 0.1$
- $\eta = 0.01$ (pomalšie učenie)

Experiment	LR	Momentum	Architektúra
XOR (A)	0.1	0.0	2–4–1
XOR (B)	0.1	0.3	2–4–4–1
XOR (C)	0.01	0.3	2–4–4–1
AND	0.1	0.0	2–4–1
AND + momentum	0.1	0.3	2–4–4–1
OR	0.1	0.0	2–4–1
OR + momentum	0.1	0.3	2–4–4–1

Table 1: Prehľad experimentálnych konfigurácií.

4 Výsledky a porovnanie

V tejto časti uvádzam všetky grafy tréningovej chyby a dopĺňam k nim analýzu.

4.1 XOR – 2–4–1, LR = 0.1, momentum = 0.0

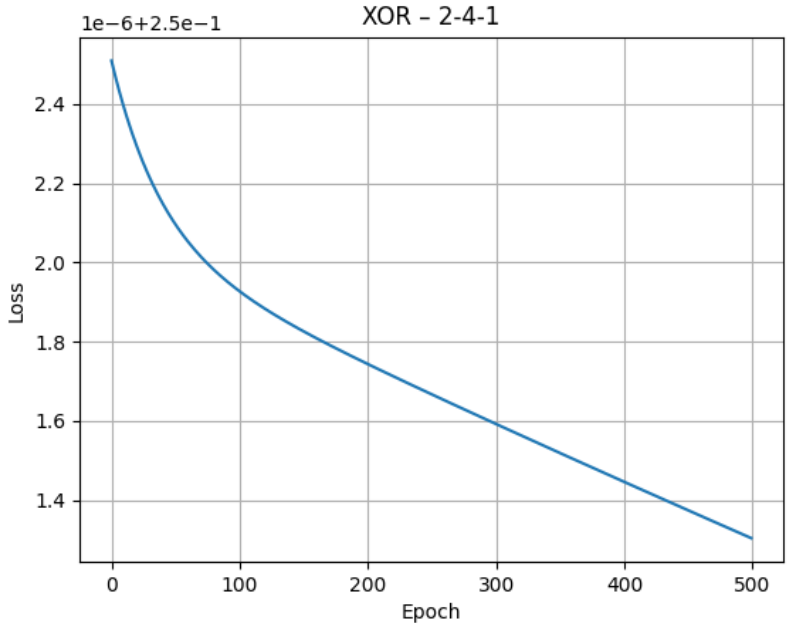


Figure 1: Priebeh tréningovej chyby – XOR 2–4–1.

Pozorovanie: Chyba ostáva prakticky konštantná (0.25). Jednovrstvový perceptrón nedokáže aproximovať XOR, čo potvrdzuje teóriu (XOR nie je lineárne separovateľný).

4.2 XOR – 2–4–4–1, LR = 0.1, momentum = 0.3

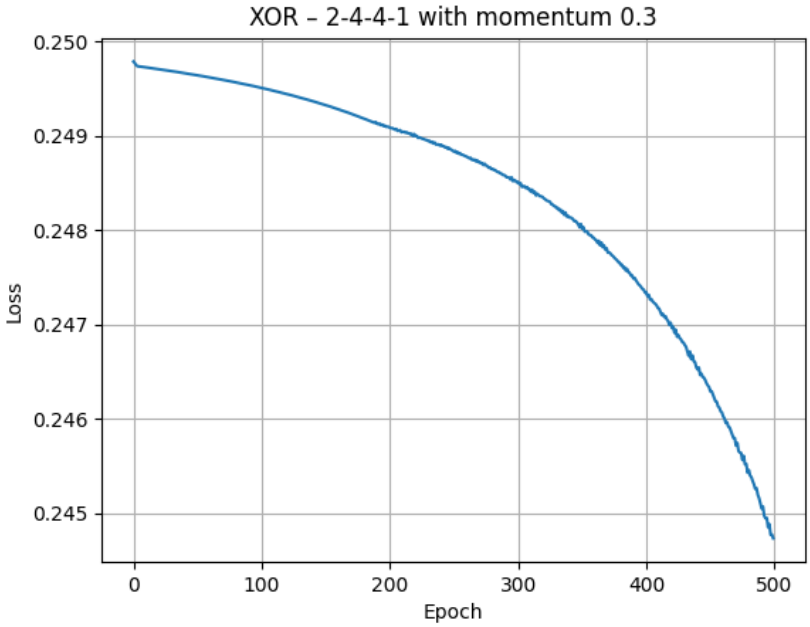


Figure 2: Priebeh tréningovej chyby – XOR 2–4–4–1, momentum 0.3

Pozorovanie: Chyba postupne klesá. Momentum zrýchľuje konvergenciu a sieť sa učí výrazne lepšie než pri jednej skr. vrstve. Predikcie sa približujú správnym hodnotám.

4.3 XOR – vplyv learning rate

Nastavenie	Finálna loss	Konvergencia
LR = 0.1	0.245	rýchla
LR = 0.01	0.2495	pomalá, stagnuje

Table 2: Vplyv rýchlosti učenia na problém XOR.

Záver: Nižší learning rate spôsobuje pomalšie učenie a v XOR dokonca stagnáciu.

4.4 AND – 2–4–1 a 2–4–4–1

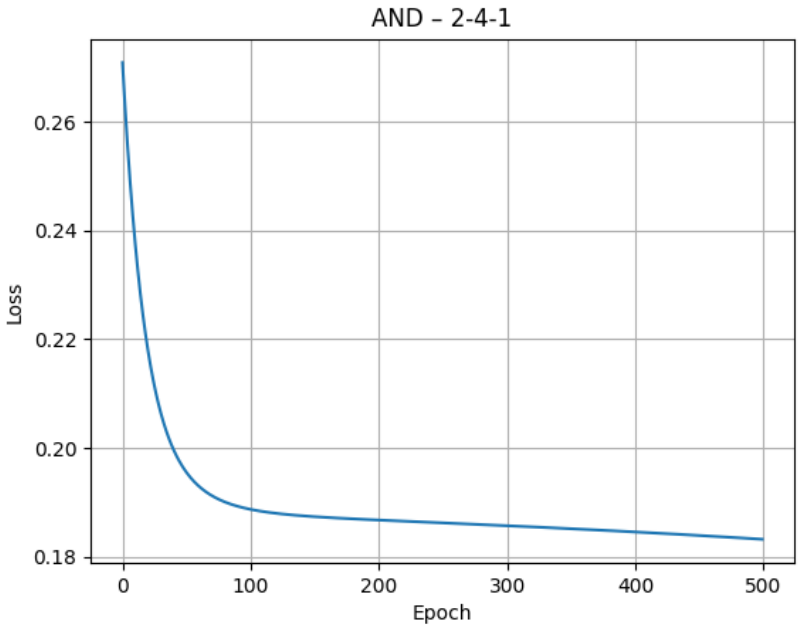


Figure 3: Priebeh chyby – AND 2–4–1

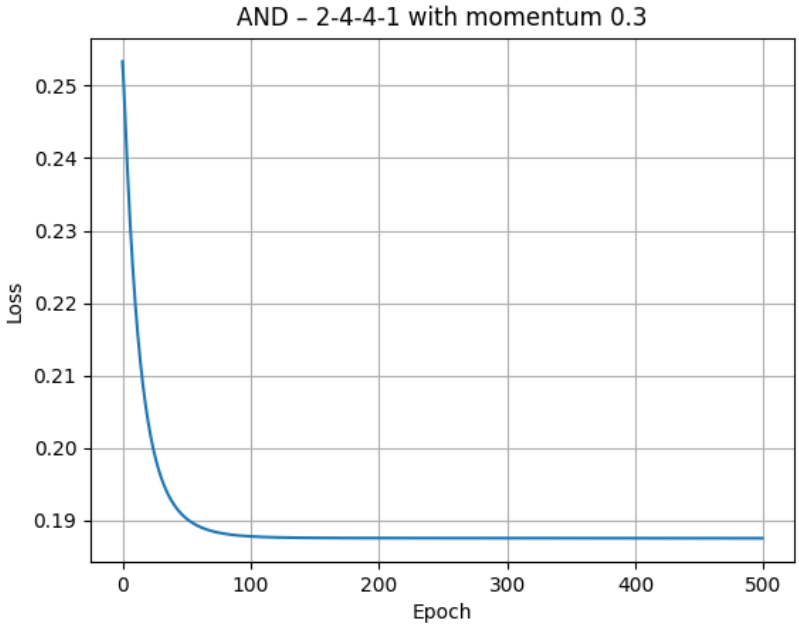


Figure 4: Priebeh chyby – AND 2–4–4–1, momentum 0.3

Pozorovanie: Keďže AND je lineárne separovateľný, aj jednoduchší model ho rýchlo naučí. Druhá skrytá vrstva už neprináša výrazné zlepšenie.

4.5 OR – 2–4–1 a 2–4–4–1

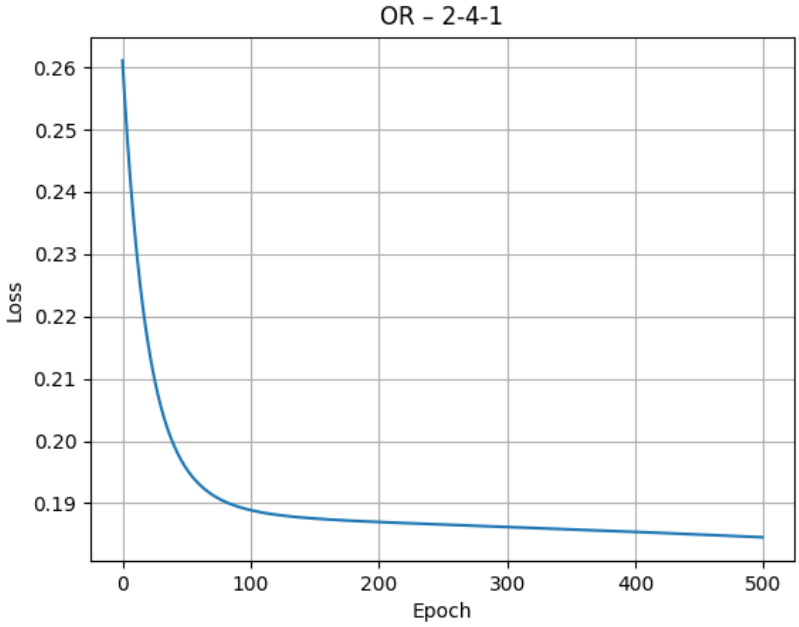


Figure 5: Priebeh chyby – OR 2–4–1

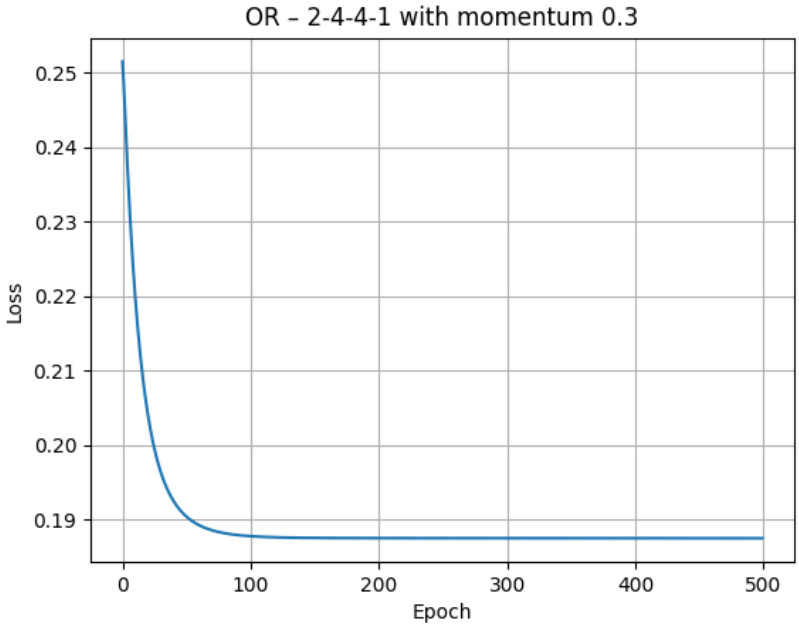


Figure 6: Priebeh chyby – OR 2–4–4–1, momentum 0.3

Pozorovanie: OR sa učí rovnako dobre v oboch architektúrach. Momentum mierne zrýchľuje konvergenciu, ale rozdiel nie je výrazný.

5 Zhodnotenie a záver

Na základe experimentov môžeme zhrnúť:

- AND a OR sú **lineárne separovateľné** – sieť ich naučí v každej architektúre.
- XOR vyžaduje **minimálne jednu skrytú vrstvu** – 2–4–4–1 sa učí dobre.
- Momentum zrýchľuje učenie pri XOR a stabilizuje gradient.
- Pri príliš nízkom learning rate dochádza k **stagnácii**.

Najlepšie výsledky boli dosiahnuté pri konfigurácii:

$$\textbf{XOR – 2–4–4–1, LR = 0.1, momentum = 0.3}$$

Backpropagation bol implementovaný korektne, všetky grafy zodpovedajú očakávanému správaniu modelu a rozdiely medzi architektúrami sú jasne pozorovateľné.